SMART CCTV AI

Presentation Script for 3 Team Members

College	Priyadarshini Bhagwati College of Engineering, Nagpur
Department	Information Technology
Session	2026 – 2027
Project Guide	Dr. Manoj Chaudhari
Project Mentor	Anant Jain
Team Members	Uday Vairalkar, Piyush Chiwande, Krishna Surve, Anuj Hiwarkar, Yash Bahoriya

Presentation Distribution

Member	Slides	Topics
Member 1	1 – 4	Title · Introduction · Literature Survey · Aim & Objectives
Member 2	5 – 8	Module Distribution · M1 Architecture · Implementation · Working Demo
Member 3	9 – 11	Limitations · Future Work · References · Closing

SLIDE 1 — Title Page — Project Introduction — Member 1

WHAT TO SAY (English)

- Good morning respected sir, ma'am, and all my dear friends. Thank you for giving us this opportunity to present our project seminar today.
- We are the students of the Department of Information Technology at Priyadarshini Bhagwati College of Engineering, Nagpur, and this presentation is for the academic session 2026 to 2027.
- Our project is titled **Smart CCTV AI — Intelligent Multi-Camera Surveillance and Suspicious Behaviour Detection**, and it aims to make ordinary CCTV cameras smart enough to detect suspicious activity automatically.
- This project has been carried out under the valuable guidance of our project guide **Dr. Manoj Chaudhari sir**, and under the mentorship of **Anant Jain sir**, whose support has been extremely helpful throughout our work.
- Our team consists of five members — Uday Vairalkar, Piyush Chiwande, Krishna Surve, Anuj Hiwarkar, and Yash Bahoriya. Out of these, three of us will be presenting the project today.
- With your permission, I would like to begin the presentation by explaining the problem that motivated us to build this system.

BOL KAISE (Hinglish)

- *Good morning respected sir, ma'am aur mere sabhi dosto. Aap logon ne humein aaj apna project seminar present karne ka mauka diya, iske liye bahut bahut dhanyawaad.*
- *Hum Priyadarshini Bhagwati College of Engineering, Nagpur ke Information Technology department ke students hai, aur ye presentation academic session 2026-27 ke liye hai.*
- *Hamare project ka naam hai **Smart CCTV AI — Intelligent Multi-Camera Surveillance and Suspicious Behaviour Detection**, aur iska aim hai normal CCTV cameras ko itna smart banana ki wo suspicious activity ko khud detect kar sake.*
- *Ye project hamare project guide **Dr. Manoj Chaudhari sir** ki valuable guidance mein aur **Anant Jain sir** ki mentorship mein complete kiya gaya hai. Unka support hamesha bahut helpful raha.*
- *Hamari team mein paanch members hai — Uday Vairalkar, Piyush Chiwande, Krishna Surve, Anuj Hiwarkar, aur Yash Bahoriya. Inme se aaj hum teen log present karenge.*
- *Aapki permission se main presentation ki shuruaat karta hun us problem se, jo hume ye system banane ke liye motivate kiya.*

KEY POINTS TO REMEMBER

- Bolne ka order: College → Department → Session → Project → Guide → Mentor → Team.
- Guide (Dr. Manoj Chaudhari) aur Mentor (Anant Jain) ke naam clear boliye.
- Confident smile aur eye contact rakhna hai.
- Speed dhimi rakho — first slide pe rush hone se pura tone bigad jaata hai.

SLIDE 2 — Introduction — Problem & Approach — Member 1

WHAT TO SAY (English)

- Let us begin by understanding the actual problem. Traditional CCTV cameras that we see everywhere today are limited to just one function — recording video. They do not have any intelligence to understand what is happening in the video.
- Because of this, detecting any suspicious activity depends completely on human operators, who have to sit and watch many camera feeds at the same time. This is not just tiring, it is also unreliable.
- Research in this field has clearly shown that human attention drops sharply after about twenty minutes of continuous monitoring. As a result, many critical events are missed, and by the time someone reviews the recording, it is already too late.
- There are intelligent commercial solutions available in the market, but they are extremely expensive and completely proprietary, which makes them out of reach for schools, small offices, and academic use.
- To solve this problem, our project follows a four-part approach, which you can see on the right side of the slide. First, we have **Multi-Camera Ingestion**, where we capture live video from webcams, IP cameras, and video files. Second, we have **Person Detection**, where we use YOLOv8 combined with ByteTrack to detect people and give each person a unique tracking ID. Third, we have **Behaviour Analysis**, where we apply pose-based rules to identify suspicious actions such as loitering or fighting. And fourth, we have a **Web Dashboard** that provides real-time streaming along with alerts and an event log.
- In simple words, we are building a CCTV system that does not just record, but actually thinks about what it sees and alerts the operator when something wrong happens.

BOL KAISE (Hinglish)

- *Sabse pehle actual problem samajhte hai. Aaj jo bhi CCTV cameras humein har jagah dikhte hai wo sirf ek kaam karte hai — video record karna. Unme koi intelligence nahi hoti ki wo video mein kya ho raha hai wo samajh sake.*
- *Isliye kisi bhi suspicious activity ko detect karne ke liye human operator chahiye, jise ek saath itne saare camera feeds dekhne padte hai. Ye kaam thakane wala bhi hai aur reliable bhi nahi hai.*
- *Research clearly bolti hai ki 20 minute continuously monitoring karne ke baad human attention bahut drop ho jaati hai. Isliye kayi critical events miss ho jaate hai, aur jab tak koi recording review karta hai, tab tak bahut late ho chuka hota hai.*
- *Market mein intelligent commercial solutions hai, but wo bahut mehenge aur proprietary hai, jo schools, small offices ya academic use ke liye affordable nahi hai.*
- *Is problem ko solve karne ke liye hamare project ka approach 4 parts mein hai, jo aap right side pe dekh sakte hai. Pehla — **Multi-Camera Ingestion**, jisme hum webcam, IP camera aur video file se live video lete hai. Doosra — **Person Detection**, jisme YOLOv8 aur ByteTrack use karke persons detect karte hai aur har person ko unique tracking ID dete hai. Teesra — **Behaviour Analysis**, jisme pose-based rules se loitering ya fighting jaise suspicious actions*

*detect karte hai. Aur chautha — **Web Dashboard**, jo real-time streaming ke saath alerts aur event log deta hai.*

- *Simple language mein bolo to, hum ek aisa CCTV system bana rahe hai jo sirf record nahi karta, balki dekhta hai, sochta hai, aur kuch galat hone pe operator ko alert bhi karta hai.*

KEY POINTS TO REMEMBER

- Problem: human operator sab feeds ek saath dekh nahi sakta, aur 20 min baad attention girta hai.
- Commercial solutions mehenge aur proprietary hai.
- Hamara approach 4 parts mein hai: **Ingestion** → **Detection** → **Behaviour** → **Dashboard**.
- Tagline: *CCTV jo sirf record nahi karta, sochta bhi hai.*

SLIDE 3 — Literature Survey — Member 1

WHAT TO SAY (English)

- This slide presents our literature survey. Before choosing any technology, we studied the existing research and industry standards in three important areas, which are shown here in three separate columns.

- The first column, in green, is **Video Ingestion**. Here we studied **OpenCV VideoCapture** by Bradski from the year 2000, which is the standard camera interfacing API used across the industry. We also looked at the **RTSP protocol** defined in RFC 7826 from 2016, which is the most widely used streaming standard for IP cameras. And finally, we studied the **drop-oldest buffering** approach by Chen and team from 2019, which is a common real-time surveillance practice where the system prefers the most recent frames over completeness.

- The second column, in blue, is **Detection and Tracking**. Here we reviewed **YOLOv8** by Jocher and team from 2023, which is a modern one-stage object detector known for its strong CPU inference speed. We also studied **ByteTrack** by Zhang and team from ECCV 2022, which reduces the number of identity switches compared to older methods like SORT. And we looked at **DeepSORT** by Wojke and team from 2017, which gives higher accuracy but requires much more computation.

- The third column, in violet, is **Behaviour Analysis**. Here we studied **MediaPipe Pose** by Bazarevsky and team from 2020, which is the BlazePose model with 33 keypoints and works on-device. We also looked at **ST-GCN** by Yan and team from AAAI 2018, which is a graph convolutional network for action recognition but requires very large training datasets. And finally, we studied **rule-based systems** that are already deployed in industry products like BriefCam and Avigilon, which are easy to interpret but do not generalise well to new types of events.

- This detailed literature survey helped us clearly justify why we chose OpenCV, RTSP, YOLOv8, ByteTrack, MediaPipe, and a rule-based approach for our own project.

BOL KAISE (Hinglish)

- *Is slide mein hamari literature survey dikhayi gayi hai. Koi bhi technology choose karne se pehle humne 3 important areas mein existing research aur industry standards ko study kiya, jo yahan 3 alag columns mein dikhaya gaya hai.*

- *Pehla column, green colour mein, hai **Video Ingestion**. Ismein humne padha **OpenCV VideoCapture**, Bradski ne year 2000 mein banaya tha, ye camera interfacing ka standard API hai. Phir humne **RTSP protocol** dekha, RFC 7826, 2016 ka — ye IP cameras ke liye sabse zyada use hone wala streaming standard hai. Aur teesra, **drop-oldest buffering**, Chen et al ne 2019 mein propose kiya — matlab buffer full hone pe purane frames drop kar do aur latest wale rakho.*

- *Doosra column, blue colour mein, hai **Detection and Tracking**. Ismein humne **YOLOv8** dekha, Ultralytics ne 2023 mein release kiya, ye modern one-stage object detector hai jo CPU pe bhi fast chalta hai. Phir **ByteTrack**, Zhang et al ka ECCV 2022 paper, jo SORT ke muqable mein ID switches kam karta hai. Aur **DeepSORT**, Wojke et al 2017, jo accurate hai but bahut zyada computation leta hai.*

- *Teesra column, violet colour mein, hai **Behaviour Analysis**. Ismein humne **MediaPipe Pose** dekha, Bazarevsky ka 2020 ka paper, jo BlazePose 33-keypoint model use karta hai aur on-device kaam karta hai. Phir **ST-GCN**, Yan et al AAAI 2018, jo action recognition ke liye graph CNN hai but bahut bada dataset chahiye. Aur last mein **rule-based systems**, jo BriefCam aur Avigilon jaise commercial products mein use hote hai — ye interpret karne mein easy hai but naye events ke liye generalise nahi karte.*

- *Isi detailed literature survey ke basis pe humne clearly justify kiya ki apne project ke liye OpenCV, RTSP, YOLOv8, ByteTrack, MediaPipe aur rule-based approach kyun choose kiya.*

KEY POINTS TO REMEMBER

- 3 columns yaad rakho: Green = Ingestion, Blue = Detection, Violet = Behaviour.
- Har technology ka year badge pe likha hai — bolne mein easy.
- Chosen tech list: OpenCV, RTSP, YOLOv8, ByteTrack, MediaPipe, Rule-based.
- End line: *"Isi literature ke basis pe humne apni tech choose ki."*

SLIDE 4 — Aim and Objective — Member 1

WHAT TO SAY (English)

- Now let us look at our aim and the objectives that flow from that aim. The main aim of our project is written in the highlighted box at the top of the slide.
- Our aim is **to design and implement an intelligent multi-camera surveillance pipeline that detects and classifies suspicious human behaviour in real time using open-source AI components**. We chose open-source technologies very deliberately, because it makes the entire project affordable, transparent, and reproducible — which is very important for an academic project.
- To achieve this aim in a structured way, we broke it down into four clear objectives, and each objective directly corresponds to one module in our final system.
- The first objective is **Ingestion**. Our goal here is to build a threaded multi-camera capture layer that can handle webcams, IP cameras over RTSP, and pre-recorded video files, all through the same interface.
- The second objective is **Detection**. Here we integrate YOLOv8 person detection together with ByteTrack, so that the system does not just detect people, but also gives each person a persistent identity — meaning person number one stays person number one across all frames.
- The third objective is **Behaviour**. In this objective, we implement a rule-based classifier that works on centroid history and pose keypoints, so the system can flag suspicious actions like loitering, sudden running, or fighting.
- The fourth objective is **Interface**. Here we deliver a clean web dashboard with live camera streaming and an event log, so the operator can view everything in one single place.
- So these four objectives together define the complete scope of our Phase 1 implementation.

BOL KAISE (Hinglish)

- *Ab hum apna aim aur objectives dekhte hai. Hamara main aim slide ke upar highlighted box mein likha hai.*
- *Hamara aim hai — **ek intelligent multi-camera surveillance pipeline design aur implement karna, jo real time mein suspicious human behaviour ko detect aur classify kare, wo bhi open-source AI components use karke**. Open-source technologies humne deliberately choose ki, kyunki isse project affordable, transparent aur reproducible ho jaata hai — jo academic project ke liye bahut important hai.*
- *Is aim ko structured way mein achieve karne ke liye humne 4 clear objectives banaye hai, aur har objective directly ek module se correspond karta hai.*
- *Pehla objective — **Ingestion**. Yahan hamara goal hai ek threaded multi-camera capture layer banana, jo webcam, RTSP wale IP camera aur video files sab ek same interface se handle kar sake.*
- *Doosra objective — **Detection**. YOLOv8 person detection ko ByteTrack ke saath integrate karte hai, taaki system sirf persons detect nahi kare, balki har person ko persistent identity bhi de — matlab person number 1 saare frames mein person number 1 hi rahega.*

- *Teesra objective — **Behaviour**. Yahan hum rule-based classifier implement karte hai jo centroid history aur pose keypoints pe kaam karta hai, taaki system loitering, sudden running ya fighting jaise suspicious actions detect kar sake.*
- *Chautha objective — **Interface**. Yahan hum ek clean web dashboard deliver karte hai live camera streaming aur event log ke saath, taaki operator sab kuch ek jagah dekh sake.*
- *To ye chaaro objectives milke hamari Phase 1 implementation ka complete scope define karte hai.*

KEY POINTS TO REMEMBER

- Aim mein 3 keywords: **multi-camera**, **real time**, **open-source**.
- 4 objectives = 4 modules. Sequence: Ingestion → Detection → Behaviour → Interface.
- Persistent identity ka matlab: person 1 hamesha person 1 rahega.
- Aim padhne ke baad pause lo, phir objectives shuru karo.

HANDOVER TO NEXT MEMBER

English: "So till now, I have introduced the problem, walked you through our literature survey, and explained our aim and objectives. Now I would like to hand over the presentation to my teammate, who will explain the module distribution of our system, the architecture of Module 1, its implementation, and our actual working demo."

Hinglish: *"Yahan tak humne problem samjhi, literature survey dekhi aur aim aur objectives explain kiye. Ab main presentation aage badhata hun apne teammate ko, jo hamare system ka module distribution, Module 1 ki architecture, implementation aur actual working demo explain karenge."*

SLIDE 5 — Module Distribution — Member 2

WHAT TO SAY (English)

- Thank you. So our entire project is divided into six focused modules, and these modules are further planned across two phases. The reason for this division is simple — we cannot build everything in one go, so we structured the work in a logical order.
- The top row of the slide shows Phase 1, which is already implemented, marked with a green tick on each card. **Module 1 is Multi-Camera Ingestion**, which is responsible for acquiring video from multiple sources in real time. **Module 2 is Detection and Tracking**, which detects persons in each frame and maintains their identities across the video. **Module 3 is Behaviour Classification**, which uses rules and pose information to classify what the tracked persons are doing.
- The bottom row shows Phase 2, which is planned but not yet implemented, marked with an orange clock. **Module 4 is Face Recognition**, which will recognise and match known faces from a watchlist. **Module 5 is Backend plus Cloud**, which will store data, manage events, and provide remote access. And **Module 6 is Dashboard plus Analytics**, which will provide visual insights through a live dashboard.
- So we have three modules working end-to-end today, and three more planned for the next phase. Today's seminar focuses mainly on the first module, Multi-Camera Ingestion, which I will now explain in detail.

BOL KAISE (Hinglish)

- *Thank you. To hamara pura project 6 focused modules mein divided hai, aur ye modules 2 phases mein plan kiye gaye hai. Reason simple hai — ek saath sab kuch nahi ban sakta, isliye humne kaam ko logical order mein structure kiya.*
- *Slide ki upar wali row Phase 1 dikhati hai, jo already implemented hai — har card pe green tick lagi hai.* ***Module 1 hai Multi-Camera Ingestion****, jo multiple sources se real time mein video acquire karta hai.* ***Module 2 hai Detection aur Tracking****, jo har frame mein persons detect karta hai aur unki identity maintain karta hai.* ***Module 3 hai Behaviour Classification****, jo rules aur pose information use karke actions classify karta hai.*
- *Neeche wali row Phase 2 dikhati hai, jo planned hai but abhi implement nahi hua — orange clock lagi hai.* ***Module 4 hai Face Recognition****, jo watchlist se known faces recognise karega.* ***Module 5 hai Backend aur Cloud****, jo data store karega, events manage karega aur remote access dega. Aur* ***Module 6 hai Dashboard aur Analytics****, jo live dashboard ke through visual insights dega.*
- *To 3 modules end-to-end kaam kar rahe hai aaj, aur 3 modules next phase ke liye planned hai. Aaj ka seminar mainly first module, Multi-Camera Ingestion pe focused hai, jo ab main detail mein explain karta hun.*

KEY POINTS TO REMEMBER

- Total 6 modules, 2 phases.
- Green tick = Implemented. Orange clock = Planned.
- Phase 1: M1, M2, M3. Phase 2: M4, M5, M6.

- End line: *"Aaj ka focus Module 1 pe hai, jo ab main detail mein explain karta hun."*

SLIDE 6 — Module 1: Multi-Camera Ingestion & AI Vision — Member 2

WHAT TO SAY (English)

- This slide gives the complete picture of Module 1. On the left side you can see the purpose, in the middle you can see the data flow diagram, and on the right side we have three highlighted facts that summarise our design.
- The purpose of Module 1 is to acquire raw video from heterogeneous sources. The word heterogeneous simply means different types of sources — such as a laptop webcam, an IP camera over RTSP, and a pre-recorded video file. Module 1 takes video from all these sources and delivers clean, timestamped frames to the downstream AI pipeline, without blocking the pipeline at any point.
- Below the purpose, you can see four main responsibilities of this module. First, it manages the lifecycle of N camera streams, meaning it can handle any number of cameras. Second, it uses threaded capture so that the AI loop is never blocked while a camera is being read. Third, it uses a bounded queue with a drop-oldest policy to keep memory usage under control. And fourth, it does basic frame preprocessing, such as cropping, colour conversion, and resizing.
- The middle diagram shows the actual data flow. On the top we have three heterogeneous sources — a webcam, an IP camera over RTSP, and a video file. All three of them feed into our Multi-Threaded Ingestion Engine. From there, the frames go into a Bounded Frame Queue that follows the drop-oldest policy, and finally the frames are handed over to the AI Pipeline for detection and tracking.
- On the right side, the three highlighted numbers summarise this design. The number three represents the three source types we support, which are Webcam, RTSP, and File. The letter N represents any number of cameras working in parallel, using our thread-per-camera design. And the number one represents our buffer depth of one frame, which means only the latest frame is kept and older ones are dropped.
- This design keeps the pipeline real-time and stable, even when the system is under heavy load.

BOL KAISE (Hinglish)

- *Ye slide Module 1 ki puri picture dikhati hai. Left side pe purpose hai, middle mein data flow diagram hai, aur right side pe 3 highlighted facts hai jo hamare design ko summarise karte hai.*
- *Module 1 ka purpose hai heterogeneous sources se raw video acquire karna. Heterogeneous ka simple matlab hai different types ke sources — jaise laptop webcam, RTSP wala IP camera, aur pre-recorded video file. Module 1 in sab sources se video leta hai aur clean, timestamped frames downstream AI pipeline ko deliver karta hai, wo bhi bina pipeline ko block kiye.*
- *Purpose ke neeche 4 main responsibilities hai. Pehla — N camera streams ka lifecycle manage karta hai, matlab kitne bhi cameras handle kar sakta hai. Doosra — threaded capture use karta hai taaki AI loop kabhi block na ho jab camera read ho raha ho. Teesra — bounded queue with drop-oldest policy use karta hai taaki memory control mein rahe. Chautha — basic frame preprocessing karta hai jaise cropping, colour conversion aur resizing.*

- *Middle diagram actual data flow dikhata hai. Upar 3 heterogeneous sources hai — webcam, RTSP wala IP camera aur video file. Teeno ka data hamare Multi-Threaded Ingestion Engine mein jaata hai. Wahan se frames Bounded Frame Queue mein jaate hai jo drop-oldest policy follow karti hai, aur last mein frames AI Pipeline ko handover ho jaate hai detection aur tracking ke liye.*
- *Right side pe 3 highlighted numbers hai jo design ko summarise karte hai. Number 3 matlab 3 source types support karte hai — Webcam, RTSP, File. Letter N matlab kitne bhi cameras parallel mein kaam kar sakte hai, thread-per-camera design se. Aur number 1 matlab hamara buffer depth 1 frame ka hai — sirf latest frame rakhte hai, purane drop kar dete hai.*
- *Is design se pipeline real-time aur stable rehti hai, chahe system pe heavy load ho.*

KEY POINTS TO REMEMBER

- Heterogeneous = different types (webcam, RTSP, file).
- 4 responsibilities: lifecycle, threading, queue, preprocessing.
- Data flow: **3 sources** → **Ingestion Engine** → **Bounded Queue** → **AI Pipeline**.
- Numbers: **3** source types, **N** parallel cameras, **1** buffer depth.

SLIDE 7 M1 Implementation — Member 2

WHAT TO SAY (English)

- This slide shows exactly what we have actually built inside Module 1. On the left side you can see the implemented components, and on the right side you can see the technology stack we have used.
- Let me first go through the implemented components on the left. The first component is **CameraSource**, which is a class that handles capture from a single camera in its own thread, wrapping OpenCV's cv2.VideoCapture function. The second is **CameraManager**, which manages multiple CameraSource objects and gives frames back in a round-robin fashion, so each camera gets fair processing time.
- The third component is the **Preprocessor**, which provides utility functions like crop_bbox for cropping a region around a detected person, preprocess_for_pose for preparing crops for the MediaPipe pose model, and letterbox for resizing frames while keeping the aspect ratio. The fourth component is **FramePacket**, which is a simple data structure that carries a frame along with its camera ID, timestamp, and frame number, so downstream modules always know exactly which frame they are processing.
- And the fifth component is **AppConfig**, which loads our configuration from a YAML file using the Pydantic library, so all thresholds and camera settings are validated at startup.
- On the right side, we have the technology stack. We are using **Python 3.11** as our main programming language, and **OpenCV version 4.8 or newer** for camera capture and image processing. Under the hood, OpenCV uses **FFmpeg** to handle RTSP streams reliably. We use **Pydantic 2.x** for validating our configuration, and **PyYAML** to read the YAML files. For parallel camera capture we rely on Python's built-in **threading** module, and we use **queue.Queue** for our thread-safe bounded frame buffer.
- All these components are modular, independently testable, and easy to replace in future work.

BOL KAISE (Hinglish)

- *Ye slide dikhati hai ki humne Module 1 mein actually kya banaya hai. Left side pe implemented components hai, aur right side pe technology stack hai jo humne use ki.*
- *Pehle left side ke components dekhte hai. Pehla component **CameraSource** hai, ye ek class hai jo single camera ka capture apne thread mein handle karti hai aur OpenCV ke cv2.VideoCapture function ko wrap karti hai. Doosra **CameraManager** hai, jo multiple CameraSource objects manage karta hai aur round-robin fashion mein frames deta hai, taaki har camera ko fair time mile.*
- *Teesra component **Preprocessor** hai, jo utility functions provide karta hai — crop_bbox detected person ke around region crop karne ke liye, preprocess_for_pose MediaPipe pose model ke liye crops prepare karne ke liye, aur letterbox aspect ratio maintain karte hue resize karne ke liye. Chautha **FramePacket** hai, ye ek simple data structure hai jo frame ke saath uska camera ID, timestamp aur frame number bhi carry karta hai, taaki downstream modules ko pata rahe konsa frame process ho raha hai.*

- *Paanchwa component **AppConfig** hai, jo YAML file se hamari configuration load karta hai Pydantic library ka use karke, taaki saare thresholds aur camera settings startup pe hi validate ho jaaye.*

- *Right side pe hamara technology stack hai. Main language **Python 3.11** hai, aur camera capture aur image processing ke liye **OpenCV 4.8 ya newer**. OpenCV under the hood **FFmpeg** use karta hai RTSP streams handle karne ke liye. **Pydantic 2.x** configuration validate karne ke liye, aur **PyYAML** YAML files read karne ke liye. Parallel camera capture ke liye Python ka built-in **threading** module use kiya, aur **queue.Queue** thread-safe bounded frame buffer ke liye.*

- *Ye saare components modular hai, independently testable hai, aur future mein easily replace kiye ja sakte hai.*

KEY POINTS TO REMEMBER

- 5 components: CameraSource, CameraManager, Preprocessor, FramePacket, AppConfig.
- 7 techs: Python, OpenCV, FFmpeg, Pydantic, PyYAML, threading, queue.Queue.
- Round-robin = every camera gets fair turn.
- Pydantic = config validation at startup, so errors are caught early.

SLIDE 8 M1 Working Demo (4 Screenshots) Member 2

WHAT TO SAY (English)

- This slide shows the actual working demo of Module 1. You can see four screenshots taken from our running system, and each screenshot highlights a different capability of the module.
- In the top-left screenshot, we can see a single live webcam feed running inside our web dashboard. The video is playing in real time, and at the bottom you can see the current FPS is 18.7 and the resolution is 640 by 480. On the left sidebar we have the dashboard navigation with Dashboard, Live View, Events, and Settings tabs.
- In the top-right screenshot, you can see multiple camera sources being processed in parallel. There are four tiles visible — Webcam 1 running at 19.1 FPS, IP Camera over RTSP running at 18.3 FPS, Webcam 2 running at 17.6 FPS, and a video file playing at 20.2 FPS. This proves that our multi-source parallel capture design is actually working.
- In the bottom-left screenshot, we can see the source configuration page. It shows the list of all configured sources with their ID, name, type, source path, status, and action buttons. Row one is Webcam 1 at index 0, live. Row two is the IP Camera with the RTSP URL, live. Row three is Webcam 2. And row four is a sample video file from D drive, currently playing. There is also an Add Source button on the top right for adding new cameras.
- In the bottom-right screenshot, we can see the performance dashboard. The average FPS across all sources is 18.6, we have 4 sources active, only 27 frames have been dropped which is just 0.11 percent, and the buffer utilisation is 62 out of 100. Below that, there is a bar chart showing per-source FPS for each camera.
- So together, these four screenshots clearly prove that Module 1 is working reliably with multiple cameras, real-time streaming, and very low frame drops.

BOL KAISE (Hinglish)

- *Ye slide Module 1 ka actual working demo dikhati hai. Aap 4 screenshots dekh sakte ho jo running system se liye gaye hai, aur har screenshot module ki alag capability highlight karta hai.*
- *Top-left screenshot mein hum single live webcam feed dekh sakte hai jo hamare web dashboard ke andar chal raha hai. Video real time mein play ho rahi hai, aur neeche FPS 18.7 aur resolution 640x480 dikh raha hai. Left sidebar mein Dashboard, Live View, Events aur Settings tabs hai.*
- *Top-right screenshot mein multiple camera sources parallel mein process ho rahe hai. 4 tiles dikh rahi hai — Webcam 1 19.1 FPS pe, IP Camera RTSP 18.3 FPS pe, Webcam 2 17.6 FPS pe, aur video file 20.2 FPS pe play ho rahi hai. Ye prove karta hai ki hamara multi-source parallel capture design actually kaam kar raha hai.*
- *Bottom-left screenshot mein source configuration page dikh raha hai. Saare configured sources ki list hai — ID, name, type, source path, status aur action buttons ke saath. Row 1 Webcam 1 hai index 0 pe, live. Row 2 IP Camera hai RTSP URL ke saath, live. Row 3 Webcam 2. Aur row 4 D drive se sample video file, abhi playing. Upar right pe Add Source button bhi hai naye cameras add karne ke liye.*

- *Bottom-right screenshot mein performance dashboard dikh raha hai. Average FPS 18.6 hai saare sources ka, 4 sources active hai, sirf 27 frames dropped hue matlab 0.11 percent, aur buffer utilisation 62 out of 100 hai. Neeche bar chart hai per-source FPS ka.*
- *To ye 4 screenshots milke clearly prove karte hai ki Module 1 reliably kaam kar raha hai — multiple cameras, real-time streaming aur bahut kam frame drops ke saath.*

KEY POINTS TO REMEMBER

- 4 screenshots — bolte waqt slide pe point karo.
- TL = single feed. TR = multi-camera. BL = config. BR = performance.
- Yaad rakho numbers: **18.6 avg FPS, 4 sources, 0.11% dropped**.
- End line: *"Ye prove karta hai ki system reliably kaam kar raha hai."*

HANDOVER TO NEXT MEMBER

English: "Now that we have covered the module distribution, the architecture of Module 1, its implementation, and the actual working demo, I would like to hand over the presentation to my teammate, who will talk about the current limitations of the system, the planned next steps, our overall future work. and the references we studied."

Hinglish: *"Ab tak humne module distribution, Module 1 ki architecture, implementation aur actual working demo cover kar liya. Ab main presentation handover karta hun apne teammate ko, jo system ki current limitations, planned next steps, overall future work aur references explain karenge."*

SLIDE 9 — Current Limitations & Next Steps — Member 3

WHAT TO SAY (English)

- Thank you. Every honest engineering project has some limitations, and we believe it is important to accept them openly. This slide is divided into two parts — the current limitations of our system are shown on the left in red, and our planned next steps are shown on the right in green.

- Starting with the limitations on the left. First, we do not have frame synchronisation across cameras, which means each camera runs independently and their timestamps are not perfectly aligned. Second, we do not have auto-reconnect for RTSP streams — if an IP camera disconnects due to network loss, it stays disconnected until the system is restarted manually. Third, our decoding is currently CPU-only, meaning we are not using any hardware acceleration for video decoding. Fourth, the frame-drop rate is calculated internally, but it is not yet exposed as a live metric on the web dashboard. And fifth, we do not have cross-camera identity association, so a person seen on Camera 1 and Camera 2 gets two completely different tracking IDs.

- Now moving to the planned next steps on the right side, which directly address these limitations. First, we plan to add RTSP auto-reconnect with exponential backoff — meaning the system will keep trying to reconnect after gradually increasing intervals. Second, we will expose the frame-drop rate as a live metric on the web dashboard, so operators can monitor stream health. Third, we will add optional GPU-accelerated decoding using NVDEC for machines with an NVIDIA GPU. Fourth, we plan to add frame-timestamp synchronisation across cameras so all feeds stay aligned in time. And fifth, we will implement adaptive frame-rate control based on pipeline load, so the system stays smooth even under heavy processing.

- So we know exactly what our limitations are today, and each one has a clear planned fix for the next stage.

BOL KAISE (Hinglish)

- *Thank you. Har honest engineering project ki kuch limitations hoti hai, aur humein lagta hai unhe openly accept karna important hai. Ye slide 2 parts mein divide hai — left side pe red mein current limitations hai, aur right side pe green mein planned next steps hai.*

- *Pehle left side ki limitations. Pehli — cameras ke beech frame synchronisation nahi hai, matlab har camera independently chalta hai aur timestamps perfectly align nahi hote. Doosri — RTSP streams ke liye auto-reconnect nahi hai, agar network loss se IP camera disconnect ho gaya to system manually restart tak disconnected hi rahega. Teesri — hamari decoding abhi CPU-only hai, hardware acceleration nahi use kar rahe. Chauthi — frame-drop rate internally calculate hoti hai but web dashboard pe abhi live metric ki tarah dikhayi nahi jaati. Aur paanchwi — cross-camera identity association nahi hai, isliye Camera 1 aur Camera 2 pe same person ko 2 alag tracking IDs milte hai.*

- *Ab right side ke planned next steps, jo directly in limitations ko address karte hai. Pehla — RTSP auto-reconnect with exponential backoff add karenge, matlab system gradually increasing intervals ke saath reconnect karta rahega. Doosra — frame-drop rate ko web dashboard pe live metric ki tarah expose karenge, taaki operators stream health monitor kar sake. Teesra —*

NVIDIA GPU wali machines ke liye optional GPU-accelerated decoding add karenge NVDEC use karke. Chautha — frame-timestamp synchronisation across cameras add karenge taaki saare feeds time mein aligned rahe. Aur paanchwa — adaptive frame-rate control implement karenge pipeline load ke basis pe, taaki heavy processing pe bhi system smooth chale.

- *To humein exactly pata hai hamari limitations kya hai aaj, aur har ek ka clear planned fix hai next stage ke liye.*

KEY POINTS TO REMEMBER

- 5 limitations aur 5 planned fixes — one-to-one mapping.
- Limitations confidently boliye — panel honest engineering ko respect karta hai.
- Key phrase: *"Har limitation ka clear planned fix hai."*

SLIDE 10 — Future Work — Member 3

WHAT TO SAY (English)

- This slide shows our overall future work in the form of a horizontal timeline. There are four milestones marked with colour-coded numbers, and each one represents a stage in the roadmap.
- The first milestone, marked with a green Now label, is Phase 1. This includes Module 1 Ingestion, Module 2 Detection and Tracking, and Module 3 Behaviour Classification. All three of these are working end-to-end today, so Phase 1 is essentially our current status.
- The second milestone, marked with a blue Next label, is Module 4 — Face Recognition. In this stage, we plan to add FaceNet or DeepFace based watchlist matching, so the system can not only detect and track people, but also recognise specific individuals from a known list.
- The third milestone, marked with a violet Phase 2 label, is Module 5 — Backend plus Cloud. Here we plan to add a FastAPI-based backend service, with role-based access control and Docker-based deployment, which will make the system production-ready and remotely accessible.
- The fourth milestone, marked with an orange Phase 2 label, is Module 6 — Dashboard plus Analytics. In this stage, we will build a React-based user interface, add mobile alerts, historical search across events, and detailed reporting.
- So our project follows a clear staged plan — we know exactly what has been done, what is coming next, and what belongs to the final phase.

BOL KAISE (Hinglish)

- *Ye slide hamara overall future work horizontal timeline ke form mein dikhati hai. 4 milestones hai colour-coded numbers ke saath, aur har ek roadmap ka ek stage represent karta hai.*
- *Pehla milestone, green Now label ke saath, Phase 1 hai. Ismein Module 1 Ingestion, Module 2 Detection & Tracking, aur Module 3 Behaviour Classification hai. Teeno aaj end-to-end kaam kar rahe hai, to Phase 1 basically hamara current status hai.*
- *Doosra milestone, blue Next label ke saath, Module 4 Face Recognition hai. Is stage mein hum FaceNet ya DeepFace based watchlist matching add karenge, taaki system sirf persons detect aur track nahi kare, balki specific individuals ko known list se recognise bhi kare.*
- *Teesra milestone, violet Phase 2 label ke saath, Module 5 Backend + Cloud hai. Yahan hum FastAPI-based backend service add karenge, role-based access control aur Docker-based deployment ke saath, jo system ko production-ready aur remotely accessible bana dega.*
- *Chautha milestone, orange Phase 2 label ke saath, Module 6 Dashboard + Analytics hai. Is stage mein React-based user interface banayenge, mobile alerts add karenge, events ka historical search aur detailed reporting bhi add karenge.*
- *To hamara project ek clear staged plan follow karta hai — humein pata hai kya done hai, kya next aa raha hai, aur kya final phase ka part hai.*

KEY POINTS TO REMEMBER

- 4 steps — Now (Phase 1), Next (M4), Phase 2 (M5), Phase 2 (M6).

- M4 = Face, M5 = Backend, M6 = Dashboard.
- Key tech names: FaceNet, DeepFace, FastAPI, Docker, React.
- End line: *"Hamara project ek clear staged plan follow karta hai."*

SLIDE 11 References — Member 3

WHAT TO SAY (English)

- This is the last slide of our presentation. Here we have listed the ten main references that we studied during the project. These include published research papers, GitHub repositories, and industry standards, and all of them are cited in proper academic format.
- For quick understanding, we can group these references into three categories. The first category is foundational tools — this includes Bradski's OpenCV library from 2000, the IETF RTSP standard RFC 7826 from 2016, Ultralytics YOLOv8 by Jocher and team from 2023, and the supervision library by Roboflow from 2023.
- The second category is tracking algorithms. This includes ByteTrack by Zhang and team from ECCV 2022, the original SORT paper by Bewley and team from ICIP 2016, and DeepSORT by Wojke and team from ICIP 2017.
- The third category is behaviour and pose analysis. This includes BlazePose by Bazarevsky and team from the CVPR 2020 workshop, ST-GCN by Yan and team from AAAI 2018, and Sultani's Real-World Anomaly Detection paper from CVPR 2018.
- With this, we conclude our presentation on the Smart CCTV AI project. On behalf of our entire team, I would like to sincerely thank the panel and all the audience members for their attention and patience. We are now open to any questions you may have.

BOL KAISE (Hinglish)

- *Ye hamari presentation ki last slide hai. Yahan humne 10 main references list kiye hai jo humne project ke dauran study kiye. Ismein published research papers, GitHub repositories aur industry standards hai, aur sab proper academic format mein cited hai.*
- *Quick understanding ke liye in references ko 3 categories mein group kar sakte hai. Pehli category — foundational tools — ismein Bradski ki OpenCV library 2000 ki, IETF ka RTSP standard RFC 7826 (2016), Ultralytics YOLOv8 (Jocher 2023), aur Roboflow ki supervision library (2023) hai.*
- *Doosri category — tracking algorithms — ismein ByteTrack (Zhang ECCV 2022), original SORT paper (Bewley ICIP 2016), aur DeepSORT (Wojke ICIP 2017) hai.*
- *Teesri category — behaviour aur pose analysis — ismein BlazePose (Bazarevsky CVPR 2020 workshop), ST-GCN (Yan AAAI 2018), aur Sultani ka Real-World Anomaly Detection paper (CVPR 2018) hai.*
- *Isi ke saath Smart CCTV AI project pe hamari presentation complete hoti hai. Puri team ki taraf se main panel aur saare audience members ko dhanyavaad karna chahunga aapke attention aur patience ke liye. Ab hum aapke questions ke liye open hai.*

KEY POINTS TO REMEMBER

- Saare 10 references yaad karne ki zaroorat nahi — 3 categories yaad rakho.
- Categories: Foundational tools, Tracking algorithms, Behaviour aur pose.
- Line: *"Sab references proper academic format mein cited hai."*
- End with clear Thank You and open the floor for questions.

POSSIBLE PANEL QUESTIONS + ANSWERS

Eighteen realistic questions the panel may ask, based on our actual PPT.

Q1. Why did you choose OpenCV for camera capture?

Answer (English): OpenCV is the industry standard open-source computer vision library, and it supports webcams, IP cameras over RTSP, and video files all through a single simple API called cv2.VideoCapture. It also uses FFmpeg internally as its backend, which handles RTSP streams very reliably, so we get robust camera support with minimal effort.

Answer (Hinglish): *OpenCV industry ki standard open-source computer vision library hai, aur ye webcam, RTSP wale IP cameras, aur video files sab ek hi simple API — cv2.VideoCapture — se handle kar leti hai. Internally FFmpeg backend use karti hai jo RTSP streams reliably chalata hai, isliye humein robust camera support minimal effort mein mil jaata hai.*

Q2. What is RTSP and why is it important for this project?

Answer (English): RTSP stands for Real-Time Streaming Protocol and it is defined in RFC 7826. It is the dominant streaming standard used by IP cameras to deliver live video across a network. Without proper RTSP support, our system would not be able to connect to real CCTV cameras used in the industry, which would make the project unrealistic.

Answer (Hinglish): *RTSP ka matlab hai Real-Time Streaming Protocol, aur ye RFC 7826 mein defined hai. IP cameras isi protocol se network pe live video bhejte hai. Agar hamare system mein proper RTSP support nahi hota to hum industry ke real CCTV cameras se connect hi nahi kar paate, aur project realistic nahi rehta.*

Q3. Why did you use multi-threading for the camera capture?

Answer (English): Camera capture is a blocking input-output operation, meaning that reading a frame from a camera can take anywhere between 30 to 100 milliseconds. If we did this in the main loop, the AI pipeline would sit idle during that time. By giving each camera its own thread, all captures happen in parallel, and the AI loop is never blocked waiting for a frame.

Answer (Hinglish): *Camera capture ek blocking I/O operation hai, matlab ek frame read karne mein 30 se 100 millisecond lag sakte hai. Agar main loop mein ye kaam karenge to AI pipeline waste time karega. Har camera ko apne thread mein chalane se saare captures parallel mein hote hai aur AI loop kabhi frame ke liye block nahi hota.*

Q4. What is the drop-oldest queue policy and why did you choose it?

Answer (English): In our design, the buffer holds only one frame at a time. When a new frame arrives and the buffer is already full, the old frame is dropped and the new one is kept. This is a very common practice in real-time systems, because showing a two-second-old face detection to an operator is useless. We always prefer freshness over completeness.

Answer (Hinglish): *Hamare design mein buffer ek time pe sirf 1 frame rakhta hai. Naya frame aaya aur buffer already full hai, to purana frame drop kar diya jaata hai aur naya rakh liya jaata hai. Real-time systems mein ye common practice hai, kyunki 2 second purani face detection kis kaam ki hai. Hum hamesha freshness ko completeness se prefer karte hai.*

Q5. What is YOLOv8 and why did you choose the nano version?

Answer (English): YOLOv8 is a modern one-stage object detector developed by Ultralytics and released in 2023. We chose the nano version because it has only about 3.2 million parameters and can run at 15 to 25 FPS on a normal CPU, without needing a GPU. For Phase 1, it gives us the right

balance between speed and accuracy on affordable hardware.

Answer (Hinglish): *YOLOv8 Ultralytics ka modern one-stage object detector hai, 2023 mein release hua. Nano version isliye choose kiya kyunki isme sirf 3.2 million parameters hai aur normal CPU pe 15 se 25 FPS de deta hai, GPU nahi chahiye. Phase 1 ke liye ye speed aur accuracy ka accha balance deta hai affordable hardware pe.*

Q6. What is ByteTrack and how is it different from SORT?

Answer (English): ByteTrack is a multi-object tracking algorithm published at ECCV 2022. Both SORT and ByteTrack track objects across frames using motion prediction, but ByteTrack also considers low-confidence detections during the association step. This significantly reduces identity switches, which means the same person keeps the same tracking ID for a much longer time.

Answer (Hinglish): *ByteTrack ek multi-object tracking algorithm hai jo ECCV 2022 mein publish hua. SORT aur ByteTrack dono objects ko frames ke across track karte hai using motion prediction, but ByteTrack low-confidence detections ko bhi association step mein consider karta hai. Isse ID switches bahut kam ho jaate hai, matlab same person ko lambe time tak same tracking ID milti rehti hai.*

Q7. What is FPS and what is a good FPS for surveillance?

Answer (English): FPS stands for Frames Per Second, which measures how many video frames the pipeline processes in one second. For real-time surveillance applications, an FPS between 15 and 25 is generally acceptable. In our system, we currently achieve an average of around 18.6 FPS across four sources running on CPU, which is well within the acceptable range.

Answer (Hinglish): *FPS ka matlab hai Frames Per Second, matlab ek second mein pipeline kitne video frames process karti hai. Real-time surveillance ke liye 15 se 25 FPS generally acceptable hai. Hamara system CPU pe 4 sources ke saath average 18.6 FPS achieve karta hai, jo acceptable range mein hai.*

Q8. How exactly does the CameraManager work?

Answer (English): CameraManager is a controller class that creates one CameraSource object per configured camera. Each CameraSource runs in its own thread and pushes frames into a bounded queue. When the main pipeline asks for frames, CameraManager gives them in a round-robin fashion — one frame from Camera 1, then Camera 2, and so on. This ensures all cameras get fair processing time and no single camera dominates the pipeline.

Answer (Hinglish): *CameraManager ek controller class hai jo har configured camera ke liye ek CameraSource object banata hai. Har CameraSource apne thread mein chalta hai aur frames bounded queue mein push karta hai. Jab main pipeline frames maangti hai, CameraManager round-robin fashion mein deta hai — ek frame Camera 1 se, phir Camera 2 se, aisa. Isse saare cameras ko fair time milta hai aur koi single camera pipeline dominate nahi karta.*

Q9. What happens when a camera disconnects?

Answer (English): Currently, when a camera disconnects, its capture thread simply stops and no more frames are pushed for that camera. This is a known limitation, which we have honestly listed on our limitations slide. As a planned next step, we will implement RTSP auto-reconnect with exponential backoff, so the system will automatically try to reconnect after intervals like 1 second, then 2, then 4 seconds, and so on.

Answer (Hinglish): *Abhi ke liye jab camera disconnect hota hai, uska capture thread ruk jaata hai aur us camera se frames aana band ho jaate hai. Ye hamari known limitation hai jo humne honestly limitations slide pe likha hai. Planned next step ke roop mein RTSP auto-reconnect with exponential backoff add karenge, matlab system automatically 1 second, phir 2, phir 4 second baad reconnect try*

karega.

Q10. What is Pydantic and why did you use it?

Answer (English): Pydantic is a Python data validation library that automatically checks whether the data you load matches the expected structure and types. We use it in our AppConfig to load our YAML configuration file, and it validates things like FPS being an integer or source type being one of our allowed values. If something is wrong, Pydantic gives a clear error at startup, instead of crashing much later at runtime.

Answer (Hinglish): *Pydantic Python ki data validation library hai jo automatically check karti hai ki jo data aap load kar rahe ho wo expected structure aur types se match karta hai ya nahi. Hum AppConfig mein YAML config load karne aur types validate karne ke liye use karte hai — jaise FPS integer honi chahiye, source_type valid value honi chahiye. Kuch galat hua to Pydantic startup pe hi clear error deti hai, runtime pe crash nahi hota.*

Q11. What is a FramePacket and why do you need it?

Answer (English): FramePacket is a simple data structure that carries a single frame along with its metadata through the pipeline. It has four fields — camera_id to tell us which camera the frame came from, frame which is the actual image data, timestamp which is when the frame was captured, and frame_number which is the sequence number. This way, downstream modules always know exactly which frame they are processing and can order or filter them correctly.

Answer (Hinglish): *FramePacket ek simple data structure hai jo ek frame ko uski metadata ke saath pipeline mein carry karta hai. Ismein 4 fields hai — camera_id ye batata hai konse camera se frame aaya, frame actual image data hai, timestamp jab frame capture hua wo time hai, aur frame_number sequence number hai. Isse downstream modules ko exactly pata rehta hai konsa frame process ho raha hai aur wo unhe correctly order ya filter kar sakte hai.*

Q12. Why do you preprocess frames? What does the Preprocessor do?

Answer (English): AI models require input in a specific format, so raw frames from a camera cannot be sent directly to them. Our Preprocessor has three main functions — crop_bbox to crop a region around a detected person, preprocess_for_pose to prepare the crop for the MediaPipe pose model, and letterbox which resizes frames while preserving the aspect ratio, which YOLO needs to produce correct results.

Answer (Hinglish): *AI models ko input ek specific format mein chahiye hoti hai, isliye camera se raw frames directly nahi bhej sakte. Hamare Preprocessor ke 3 main functions hai — crop_bbox detected person ke around crop banane ke liye, preprocess_for_pose MediaPipe pose model ke liye crop prepare karne ke liye, aur letterbox jo aspect ratio maintain karte hue resize karta hai — YOLO ko correct results ke liye ye chahiye.*

Q13. Which modules are implemented and which are planned?

Answer (English): In Phase 1, we have already implemented Module 1 which is Multi-Camera Ingestion, Module 2 which is Detection and Tracking, and Module 3 which is Behaviour Classification. All three of these are working end-to-end today. In Phase 2, we have planned Module 4 for Face Recognition, Module 5 for Backend plus Cloud, and Module 6 for Dashboard plus Analytics, which are our future work.

Answer (Hinglish): *Phase 1 mein humne already Module 1 Multi-Camera Ingestion, Module 2 Detection & Tracking, aur Module 3 Behaviour Classification implement kar diye hai. Teeno aaj end-to-end kaam kar rahe hai. Phase 2 mein Module 4 Face Recognition, Module 5 Backend + Cloud, aur Module 6 Dashboard + Analytics planned hai, jo hamara future work hai.*

Q14. Why is GPU acceleration only planned and not already used?

Answer (English): GPU acceleration requires an NVIDIA GPU along with specialised libraries like CUDA and NVDEC. Not every deployment machine has a GPU, especially in schools, small offices, or academic setups. We built Phase 1 with a CPU-first design intentionally, so that it can run on any laptop. GPU support will be added as an optional feature in future work, mainly for high-load deployments.

Answer (Hinglish): *GPU acceleration ke liye NVIDIA GPU chahiye, aur uske saath specialised libraries jaise CUDA aur NVDEC bhi. Har machine pe GPU nahi hota, especially schools, small offices aur academic setups mein. Humne Phase 1 CPU-first design ke saath intentionally banaya taaki har laptop pe chal sake. GPU support future mein optional feature ki tarah add karenge, mainly high-load deployments ke liye.*

Q15. What is heterogeneous source support in your system?

Answer (English): Heterogeneous simply means different types. Our system supports three completely different kinds of sources — a laptop webcam which is identified by an integer index like zero, an IP camera which is accessed using an RTSP URL like rtsp colon slash slash, and a video file which is accessed through a file path. All three types work through the same CameraSource class with the exact same API, so the rest of the pipeline does not have to know or care about the source type.

Answer (Hinglish): *Heterogeneous ka matlab hai different types. Hamara system 3 completely alag types ke sources support karta hai — laptop webcam jo integer index se identify hoti hai jaise 0, IP camera jo RTSP URL se access hota hai jaise rtsp colon slash slash, aur video file jo file path se access hoti hai. Teeno types same CameraSource class se same API ke through kaam karte hai, isliye baaki pipeline ko source type ke baare mein sochne ki zaroorat nahi.*

Q16. Why did you choose open-source AI components?

Answer (English): There are three main reasons. First, cost — commercial solutions like BriefCam and Avigilon are extremely expensive and not affordable for academic or small-scale deployments. Second, transparency — we can read the actual code and understand exactly what each model is doing internally. Third, reproducibility — anyone can install our project and run it without paying for a licence, which is very important for academic research and evaluation.

Answer (Hinglish): *Iske 3 main reasons hai. Pehla — cost, BriefCam aur Avigilon jaise commercial solutions bahut mehenge hai aur academic ya small-scale deployments ke liye affordable nahi hai. Doosra — transparency, hum actual code padh sakte hai aur exactly samajh sakte hai ki har model internally kya kar raha hai. Teesra — reproducibility, koi bhi hamara project install karke bina licence ke run kar sakta hai, jo academic research aur evaluation ke liye important hai.*

Q17. What is the difference between Module 1 and Module 2?

Answer (English): Module 1 is only about ingestion — its job is to bring raw video into the system in a clean and reliable form. It does not perform any AI processing itself. Module 2 then takes the frames produced by Module 1 and runs the AI part — it detects people in each frame using YOLOv8 and tracks their identities across frames using ByteTrack. In short, Module 1 delivers frames, and Module 2 turns those frames into detections with tracking IDs.

Answer (Hinglish): *Module 1 sirf ingestion ke baare mein hai — iska kaam hai raw video ko clean aur reliable form mein system mein laana. Ye khud koi AI processing nahi karta. Module 2 phir Module 1 ke diye frames leta hai aur AI part run karta hai — YOLOv8 se har frame mein persons detect karta hai aur ByteTrack se unki identities across frames track karta hai. Short mein, Module 1 frames deta hai, Module 2 un frames ko detections with tracking IDs mein convert karta hai.*

Q18. What is the biggest strength of your project?

Answer (English): The biggest strength of our project is its modular architecture. Each module is completely independent, individually testable, and can be easily swapped out or upgraded in future work. In addition, our entire system runs on CPU using open-source components, which makes it affordable and accessible. And most importantly, Module 1 already handles real multi-camera load with less than 0.2 percent frame drops in our tests, which shows that the design actually works in practice.

Answer (Hinglish): *Hamare project ki sabse badi strength iski modular architecture hai. Har module completely independent hai, individually testable hai, aur future mein easily swap ya upgrade kiya ja sakta hai. Uske alawa, hamara pura system CPU pe chalta hai open-source components ke saath, isliye affordable aur accessible hai. Aur sabse important, Module 1 already real multi-camera load handle karta hai hamare tests mein 0.2 percent se kam frame drops ke saath, jo prove karta hai ki design practical mein bhi kaam karta hai.*

3-MEMBER PRESENTATION CHEAT SHEET

Member	Slides	Main Topics
Member 1	1, 2, 3, 4	Title · Introduction · Literature Survey · Aim & Objectives
Member 2	5, 6, 7, 8	Module Distribution · M1 Purpose & Architecture · Implementation · Workin
Member 3	9, 10, 11	Limitations & Next Steps · Future Work · References + Closing

MEMBER 1 — Opens the presentation and sets up the problem context

- Greet the panel confidently, speak slowly, and maintain eye contact throughout the introduction.
- Introduce college, department, session, project title, guide, mentor, and team members in this exact order.
- Explain the problem clearly — traditional CCTVs only record, and human attention drops after about twenty minutes.
- Cover the four-part approach: Ingestion, Detection, Behaviour, and Dashboard.
- Walk through the three literature survey columns — Green for Ingestion, Blue for Detection, Violet for Behaviour.
- Read the aim slowly, pause, and then present the four objectives as a clean flow.
- End with a smooth handover line to Member 2 before stepping back.

MEMBER 2 — Delivers the technical core of the presentation

- Explain how the project is split into six modules across two phases, with Phase 1 already implemented.
- Do a deep dive into Module 1 covering its purpose, four responsibilities, and data flow diagram.
- Highlight the three key numbers — three source types, N cameras in parallel, and buffer depth of one.
- Walk through the five implemented components: CameraSource, CameraManager, Preprocessor, FramePacket, and AppConfig.
- Present the technology stack — Python, OpenCV, FFmpeg, Pydantic, PyYAML, threading, and queue.Queue.
- Present the four demo screenshots by clearly pointing to each corner — top-left, top-right, bottom-left, bottom-right.
- Emphasise the numbers from the performance page: 18.6 average FPS, 4 active sources, only 0.11 percent dropped frames.

MEMBER 3 — Closes the presentation and handles the wrap-up

- Present the limitations honestly and calmly — the panel respects honest engineering.
- For each limitation on the left, mention the corresponding planned fix on the right.
- Walk through the future work timeline — Now (Phase 1 done), Next (M4), Phase 2 (M5), Phase 2 (M6).
- Remember the key technology names: FaceNet, DeepFace, FastAPI, Docker, and React.
- Group the ten references into three categories: Foundational tools, Tracking algorithms, and Behaviour or pose analysis.
- End the presentation with a clear thank you and formally invite questions from the panel.
- If a technical question comes, hand it over to the member who actually presented that section.

QUICK PRESENTATION TIPS

- Practice at least two complete run-throughs of the presentation before the actual seminar, ideally in front of a small audience.
- While speaking, point at the slide with your hand — do not just stare at it, and never turn your back to the panel.
- Speak slowly and clearly. Nervousness makes people rush, so consciously take small breaths between sentences.
- Do not memorise the script word-for-word — memorise the flow, and speak naturally in your own tone.
- If you forget a point, calmly move to the next one. Do not panic and do not apologise.
- During questions, let the person who presented that section answer first, and only jump in if they need help.
- If you truly do not know an answer, say honestly: "That is a very good question, sir. We have not explored that specific angle yet, but it can be part of our future work."